

AI STUDENT NOTEBOOK WEBSITE

February 21, 2026

Group 10

CS 4398

Software Engineering Project

Spring 2026

Khang Ngo

Jay McCall

Anna Mason

TABLE OF CONTENT

1. INTRODUCTION	3
1.1. PURPOSE OF THE SRS	3
1.2. PROBLEM STATEMENT	3
2. FUNCTIONAL REQUIREMENTS	4
2.1. UPLOAD NOTES	4
2.2. ORGANIZE NOTES INTO CLASSES	4
2.3. GENERATE AI SUMMARIES	4
2.4. GENERATE SIMPLIFIED EXPLANATIONS	4
2.5. CREATE REVIEW QUESTIONS (ACTIVE RECALL)	4
2.6. OCR FOR HANDWRITTEN NOTES	4
2.7. AI CHAT FOR CLARIFICATION	4
2.8. CALENDAR / REMINDERS	4
2.9. UML CLASS DIAGRAM	5
3. GUI	6
3.1. WEBPAGE MOCKUP (WIREFRAME):	6
3.1.1. Sign In Page	6
3.1.2. Notebook Page	6
3.1.3. Calendar Page	7
3.1.4. Quiz Page	7
4. NON-FUNCTIONAL REQUIREMENTS	8
4.1. PERFORMANCE	8
4.2. SCALABILITY	8
4.3. RELIABILITY & AVAILABILITY	8
4.4. SECURITY	8
4.5. USABILITY	8
4.6. MAINTAINABILITY	8
4.7. COMPLIANCE	8
5. IMPLEMENTATION CONSTRAINTS	9
5.1. FREE TIER OPENAI API KEY RATE LIMITS	9
5.2. LARGE FILE SIZE	9
5.3. OCR AND TEXT EXTRACTION	9
5.4. USER'S PRIVACY AND SECURITY	9
6. REFERENCES	10

1. INTRODUCTION

1.1. Purpose of the SRS

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements of the AI Student Notebook System. This document provides a clear description of the system's intended behavior, constraints, and major features so that all team members and stakeholders share a consistent understanding of what will be developed. The SRS will guide the design, implementation, and testing phases of the project and will serve as a reference throughout the system's development lifecycle.

1.2. Problem Statement

Students often accumulate large amounts of notes across multiple courses, stored in various formats such as typed documents, PDFs, scanned pages, and handwritten images. Organizing this material, locating important information, and studying efficiently can be challenging and time-consuming. Traditional digital note-storage tools provide basic organization but offer limited support for comprehension, summarization, or active recall.

The AI Student Notebook System addresses these issues by providing a centralized platform where students can upload, structure, and review their notes with the assistance of AI tools. Through features such as OCR text extraction, AI-generated summaries, simplified explanations, and optional review-question generation, the system aims to help students better understand their course material and study more effectively.

2. FUNCTIONAL REQUIREMENTS

2.1. Upload Notes

The system shall allow users to upload notes in PDF, image, or text format. Users can add files individually or in batches. Uploaded notes will be stored in the user's workspace for later retrieval.

2.2. Organize Notes into Classes

Users shall be able to create class folders and place uploaded notes into them. Folders can be renamed or deleted. This organization helps students group material by course.

2.3. Generate AI Summaries

The system shall provide automated summarization for any uploaded document. Summaries will highlight key points to support quick review and comprehension.

2.4. Generate Simplified Explanations

Users may request a simplified explanation of difficult concepts within a document. The system shall return clearer, easier-to-understand versions of selected material.

2.5. Create Review Questions (Active Recall)

The system shall generate multiple-choice or short-answer questions based on a selected document. These questions help students practice active recall and test comprehension.

2.6. OCR for Handwritten Notes

When users upload handwritten images, the system shall convert the image into searchable text using OCR technology. Converted text will be stored alongside the original image.

2.7. AI Chat for Clarification

The system shall provide a chat interface where students can ask questions about their uploaded notes. Responses shall be grounded in the student's own materials.

2.8. Calendar / Reminder Integration

Users shall be able to view important academic dates and optionally set study reminders. The system may integrate note review with scheduled events.

2.9. UML Class Diagram

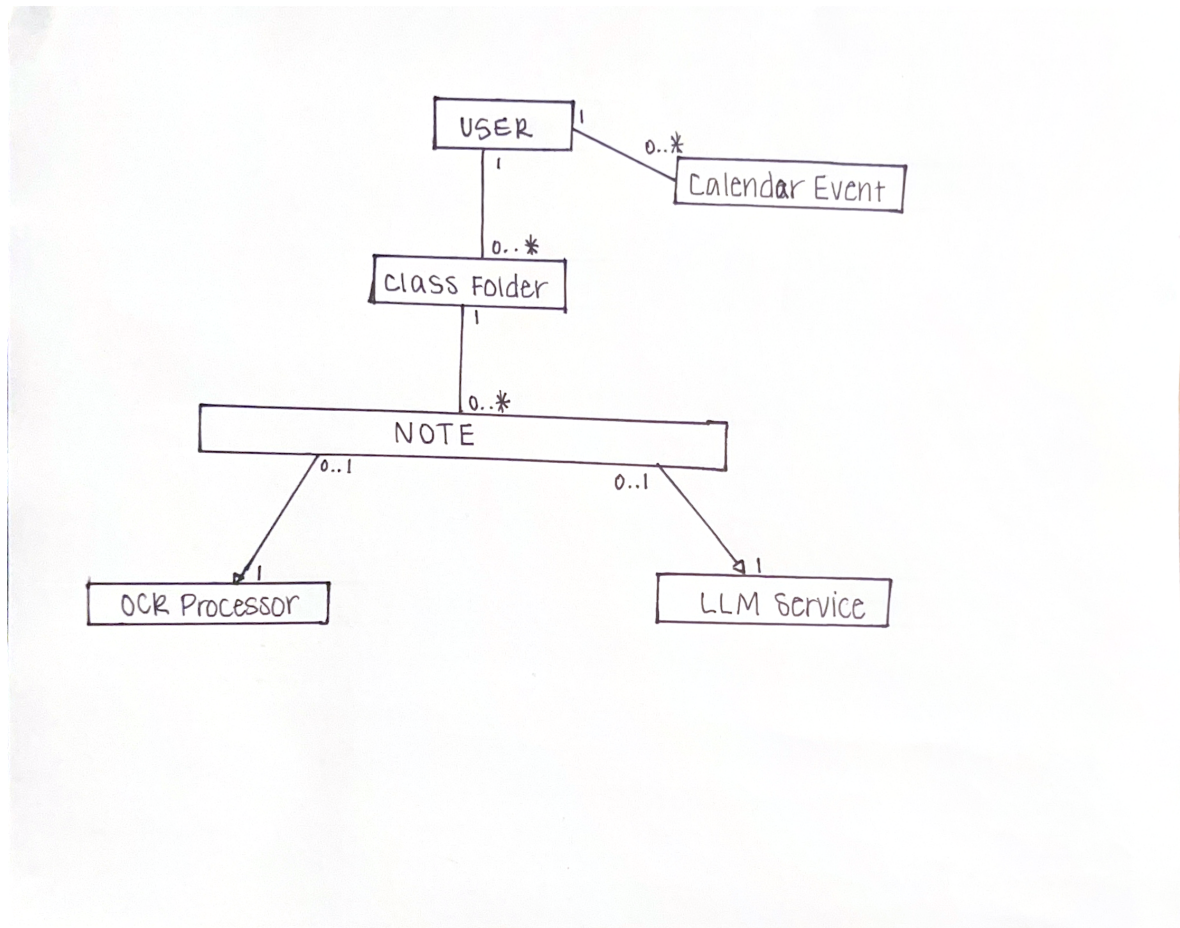


Figure 1. UML Class Diagram for the AI Student Notebook System

3. GUI

3.1. Webpage Mockup (Wireframe):

Below are conceptual designs of the individual web pages that will be displayed to the user upon request.

3.1.1. Sign In Page

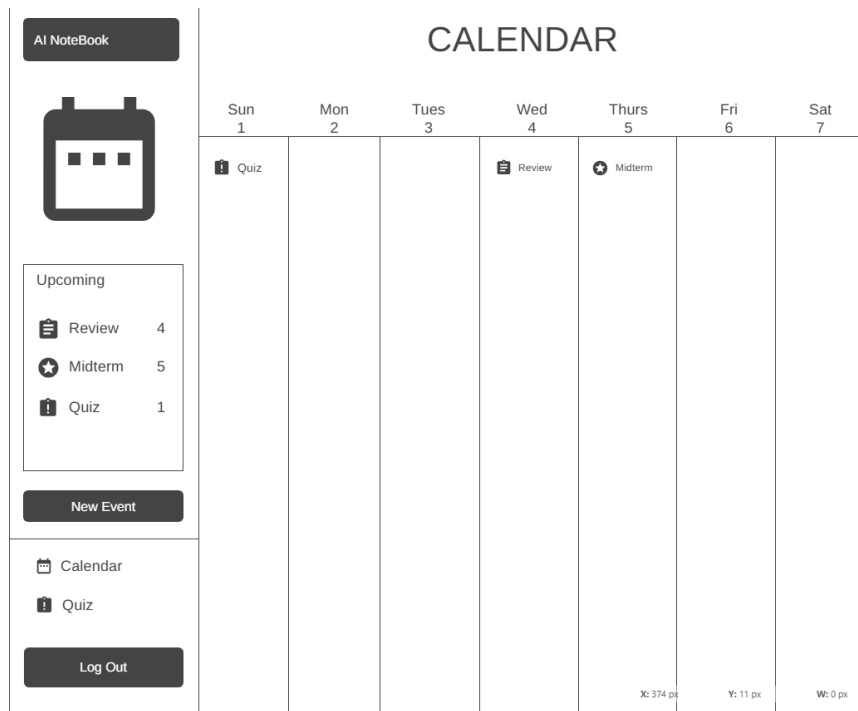
```

graph TD
    A[AI NOTEBOOK] --> B[User Icon]
    B --> C[Password Icon]
    C --> D[Log In]
    D --> E[Sign Up]
  
```

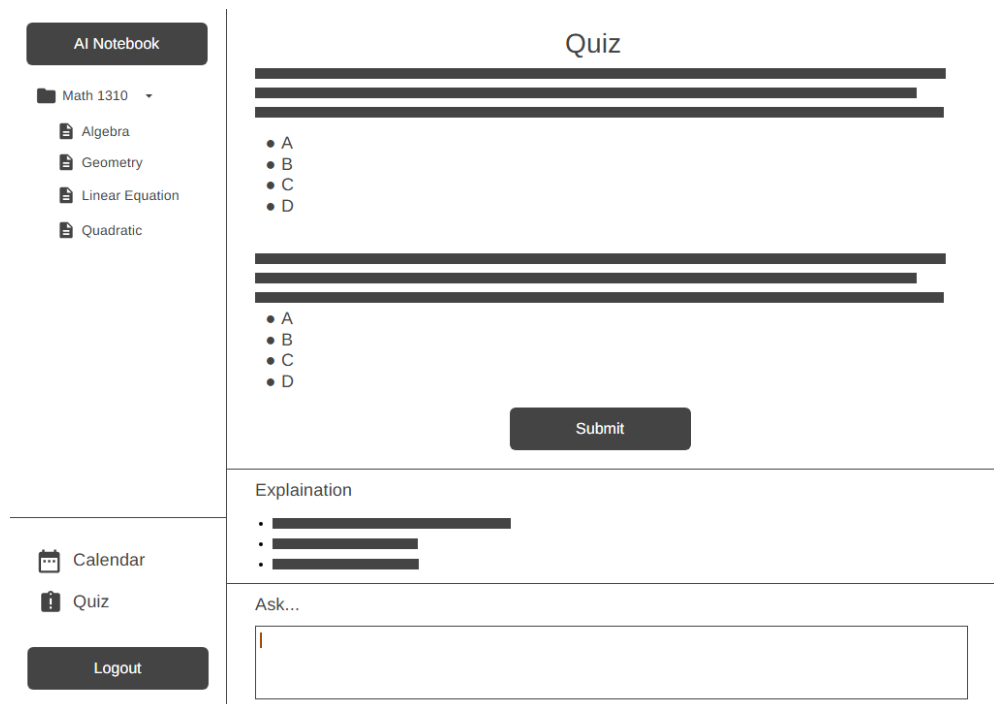
3.1.2. Notebook Page

[illegible]

3.1.3. Calendar Page



3.1.4. Quiz Page



4. NON-FUNCTIONAL REQUIREMENTS

4.1. Performance

The system must return LLM-generated responses (summaries, quiz questions, chat replies) within 10 seconds under normal network conditions. The UI must remain responsive during API calls, displaying loading indicators, so users are not left without feedback.

4.2. Scalability

The system must support at least 100 concurrent users without degradation in response time. The folder and document storage architecture must accommodate growth in the number of classes, topics, and documents per user over a full academic semester.

4.3. Reliability & Availability

The system must maintain 99.5% uptime during peak academic weeks (e.g., midterms and finals). In the event of an API failure or timeout, the system must easily handle the error and notify the user without corrupting or losing any stored documents or summaries.

4.4. Security

All user data, including uploaded documents and generated summaries, must be stored securely and accessible only by the owning user. API keys used for OpenAI communication must never be exposed to the client side and must be managed on the server side.

4.5. Usability

The interface must be intuitive enough that a new user can upload a document and generate a summary within their first session without any external guidance. The system must be accessible on both desktop and mobile browsers without loss of core functionality.

4.6. Maintainability

The codebase must follow a modular architecture separating frontend, backend, and LLM service logic, so that any one layer can be updated or replaced independently. Code must be documented sufficiently for a developer unfamiliar with the project to understand and extend any module.

4.7. Compliance

The system must comply with FERPA guidelines by ensuring that student academic materials are not shared with or used to train third-party models without explicit user consent.

5. IMPLEMENTATION CONSTRAINTS

5.1. Free Tier OpenAI API Key Rate Limits

The free tier of OpenAI only allows for 20 requests per minute and 40,000 tokens per minute of usage. Upon exceeding the limit, the program will force users to wait in a queue before they can continue to access additional requests or summarization.

5.2. Large File Size

Students' written notes may be large and may exceed hosting platform limits as well as token limits. Uploaded files should be stored in user's own storage systems where possible. Only the AI summary of each note will be stored in the database. Additionally, the system must break large notes and documents into chunks with incremental summarization.

5.3. OCR and Text Extraction

User written notes may be incorrectly translated from image to text using Optical Character Recognition (OCR). Users will be able to view the extracted text and make changes, and this version will replace the one extracted by the OCR.

5.4. User's Privacy and Security

FERPA guidelines must be followed with sensitive educational materials. Only necessary data should be transmitted to OpenAI API, communication between client, server, and APIs must be encrypted, and uploaded files must be protected from unauthorized access.

6. REFERENCES

- Wireframe.cc: Webpage design mockup